

ILP Models and Column Generation for the Minimum Sum Coloring Problem

Fabio Furini¹

*Université Paris-Dauphine, PSL Research University, CNRS, LAMSADE, 75016,
Paris, France.*

Enrico Malaguti^{2,3}

*Dipartimento di Ingegneria dell'Energia Elettrica e dell'Informazione
Università di Bologna, Bologna, Italy*

Sébastien Martin⁴

LCOMS, Université de Lorraine, 57045 Metz, France.

Ian-Christopher Ternier⁵

*Université Paris-Dauphine, PSL Research University, CNRS, LAMSADE, 75016,
Paris, France.*

Abstract

We study two Integer Linear Programming (ILP) formulations for the Minimum Sum Coloring Problem (MSCP). The problem is an extension of the classical Vertex Coloring Problem in which each color is represented by a positive natural number. The MSCP asks to minimize the sum of the cardinality of subsets of vertices receiving the same color, weighted by the index of the color, while ensuring that vertices linked by an edge receive different colors. The first ILP formulation has a polynomial number of variables while the second one has an exponential number of variables and is tackled via column generation. Computational tests show that the linear programming relaxation of the second formulation provides tight lower bounds which allow us to solve to proven optimality some hard instances of the literature.

Keywords: Graph Coloring, Integer Programming, Column Generation.

1 Introduction

Given an undirected graph $G = (V, E)$, with $|V| = n$ vertices and $|E| = m$ edges, a coloring C of G is a partition of V into k stable sets $C = \{V_1, \dots, V_k\}$ where all vertices belonging to V_i are colored with the same color i ($i = 1, \dots, k$). Given a coloring $C = \{V_1, \dots, V_k\}$ of G , the *sum coloring* of C is given by $\psi(G, C) = \sum_{i \in \{1, \dots, k\}} (i \cdot |V_i|)$. The *Minimum Sum Coloring Problem* (MSCP) consists of finding a coloring C with the minimum cost $\psi(G, C)$. This minimum cost has been denoted as $\Sigma(G)$ and called the *chromatic sum* of the graph G in the literature [9]. Finally, the smallest number of colors (or equivalently stable sets) associated with the chromatic sum $\Sigma(G)$ is called the *strength* of the graph and denoted by $s(G)$. The MSCP corresponds to the scheduling problem of incompatible unitary length jobs on identical parallel-machines, where each job has a due date equal to zero, no preemption is allowed, and the objective is to determine a schedule so that the total tardiness is minimized. Two jobs are incompatible if they cannot be executed at the same time. A related scheduling problem has been addressed in [8].

The MSCP has been introduced in Kubicka and Schwenk [9] where it is shown that problem is NP-hard for arbitrary graphs. Several articles proposed lower bounds for both $\Sigma(G)$ and $s(G)$. Alavi et al. [1] proved that $\lceil \sqrt{8m} \rceil \leq \Sigma(G) \leq \lceil \frac{3}{2}(m+1) \rceil$ holds for any graph G . Sen et al. [16] studied a generalization of the MSCP where each color has a given cost. Mitchem and Morriss [14] proved that $s(G) \leq \Delta(G) + 1$ holds where $\Delta(G)$ is the maximum degree of the vertices of G . Salavatipour [15] studied the MSCP and its variants, and proved complexity results for specific classes of graphs. Moukrim et al. [13] proposed instead a range of lower bounds for the MSCP based on clique decompositions and “partial” graphs.

Many heuristic algorithms have also been developed over the years to find good upper bounds for the MSCP. Li et al. [10] developed greedy algorithms adapting the Vertex Coloring Problem greedy algorithms DSATUR and RLF.

¹ Email: `fabio.furini@dauphine.fr`

² Supported by the Air Force Office of Scientific Research under award number FA9550-17-1-0025

³ Email: `enrico.malaguti@unibo.it`

⁴ Email: `sebastien.martin@univ-lorraine.fr`

⁵ Email: `ianchristopher.ternier@dauphine.fr`

Local search algorithms have been developed as well in [4]. Wu and Kao [18,19] devised a sophisticated greedy algorithm combined with tabu search especially effective for large graphs. Different genetic algorithms and tabu search algorithms have been developed in [5,7].

The current most successful algorithm for lower and upper bounds on the MSCP is called HESA and it has been proposed by Jin and Hao [7]; it is an algorithm based on stochastic hybrid evolutionary search. We address the interested reader to Jin et al. [6] for more information on these algorithms and a complete survey on the topic.

To the best of our knowledge, very little attention has been given to effective ILP formulations for the MSCP. The only ILP formulation has been proposed in [16] but no computational results are reported. For this reason in this manuscript we study and test two Integer Linear Programming (ILP) formulations for the MSCP. The first is the ILP formulation with a polynomial number of variables proposed in [16]; while the second new formulation has an exponential number of variables and it is tackled via column generation.

The remainder of the paper is organized as follows. In Section 2 we present the ILP models and in Section 3 we report on computational experiments performed on two sets of coloring benchmark instances from the literature. Finally, in Section 4 we depict further promising lines of research on the topic.

2 ILP Formulations

Let us denote by k an upper bound on the strength of the graph $s(G)$, the first ILP model is a compact formulation with $k \cdot n$ variables and $m + n$ constraints.

2.1 Compact Formulation

The compact formulation, called MIP_C in the following, uses binary variables x_v^i associated with each vertex $v \in V$ and each possible color $i \in \{1, \dots, k\}$:

$$Z(MIP_C) = \min \sum_{i \in \{1, \dots, k\}} \sum_{v \in V} i \cdot x_v^i \quad (1)$$

$$x_u^i + x_v^i \leq 1 \quad (u, v) \in E, i \in \{1, \dots, k\}, \quad (2)$$

$$\sum_{i \in \{1, \dots, k\}} x_v^i = 1 \quad v \in V, \quad (3)$$

$$x_v^i \in \{0, 1\} \quad v \in V, i \in \{1, \dots, k\}. \quad (4)$$

The meaning of these variables is the following: $x_v^i = 1$ if the vertex v takes color i and 0 otherwise. The objective function (1) corresponds to the sum of the costs i of each color times the number of vertices receiving the color. Constraints (2) ensure that for each edge of G and for each color $i \in \{1, \dots, k\}$, at most one of the endpoints of the edge receives the color i . Constraints (3) impose each vertex is colored with exactly one color. These partitioning constraints (“=”) can be replaced by covering constraints (“ $\geq$ ”) since the sense of the objective function imposes that each vertex is colored by exactly one color (i.e., if a vertex takes more than one color the objective function increases). Finally, constraints (4) define the variables.

To the best of our knowledge, the computational performance of MIP_C has not been tested yet. It is worth mentioning that the cost of each color is different. This fact makes MIP_C less sensitive to symmetry problems which are one of the main drawbacks of this formulation when used to solve the classical Vertex Coloring Problem (VCP). In Section 3, we computationally prove that MIP_C is capable of solving to optimality many MSCP instances of the literature.

2.2 Extended Formulation

We now introduce the new extended formulation (called MIP_E) for the MSCP with an exponential number of variables and $k + n$ constraints. Let us denote the collection of all stable sets of G as:

$$\mathcal{S} = \{S \subseteq V : (u, v) \notin E \quad \forall u, v \in S\}.$$

This model uses binary variables ξ_S^i associated with each stable set $S \in \mathcal{S}$ and each color $i \in \{1, \dots, k\}$. The variable ξ_S^i takes value one if the stable set $S = V_i$ and 0 otherwise. The new extended formulation reads as follows:

$$Z(MIP_E) = \min \sum_{i \in \{1, \dots, k\}} \sum_{S \in \mathcal{S}} c_S^i \xi_S^i \quad (5)$$

$$\sum_{S \in \mathcal{S}} \xi_S^i \leq 1 \quad i \in \{1, \dots, k\}, \quad (6)$$

$$\sum_{i \in \{1, \dots, k\}} \sum_{S \in \mathcal{S}: v \in S} \xi_S^i \geq 1 \quad v \in V, \quad (7)$$

$$\xi_S^i \in \{0, 1\} \quad i \in \{1, \dots, k\}, S \in \mathcal{S}, \quad (8)$$

where $c_S^i = i \cdot |S|$. In the objective function (5), the cost c_S^i is paid if the stable set S receives color i . Constraints (6) impose each color i is assigned to

at most one stable set S . Constraints (7) impose each vertex v is contained in at least one stable set S that receives a color (covering version). Finally, constraints (8) define the variables. The MIP_E can also be seen as a Dantzig-Wolfe decomposition of (2)-(4).

Formulation MIP_E is inspired by the classical extended formulation for VCP introduced by Mehrotra and Trick [12] which is one of the most effective formulation for the VCP. We adress the interested reader to Malaguti and Toth [11] and Cornaz, Furini and Malaguti [2] for a discussion on this topic. By replacing constraints (8) with the following ones:

$$\xi_S^i \geq 0 \quad i \in \{1, \dots, k\}, S \in \mathcal{S}, \quad (9)$$

we obtain the Linear Programming relaxation of MIP_E , denoted as LP_E . Since MIP_E has exponentially many ξ_S^i variables, *Column Generation* (CG) techniques are necessary to efficiently solve LP_E . Let us denote by π_i the dual variables associated with Constraints (6) and by μ_v the dual variables associated with Constraints (7). Then the reduced cost of a variable ξ_S^i is:

$$\tilde{c}_S^i = c_S^i - \left(\sum_{v \in S} \mu_v - \pi_i \right) = i \cdot |S| - \sum_{v \in S} \mu_v + \pi_i. \quad (10)$$

Hence, to solve LP_E by CG, we initialize the model with a subset of variables containing a feasible solution, and, given an optimal primal-dual solution $(\xi^*, (\pi^*, \mu^*))$ to this restricted master problem, we iteratively add variables with negative reduced cost. For a given color i , this pricing problem can be modeled as a Maximum Weight Stable Set Problem on G , where the weight of a vertex v is defined as $\mu_v^* - i$. Clearly, k pricing problems have to be solved at each iteration of the CG scheme, one for each possible color. When no stable set with negative reduced cost exists, the rounded-up value of the optimal solution value of LP_E provides a valid lower bound for the MSCP.

In order to speed up the convergence of the CG procedure we propose the following two ideas. By solving the CG pricing problems for increasing color indexes, when a negative reduced cost variable ξ_S^i is found for a color i , we add to the current restricted master problem the variables ξ_S^j for $j = i+1, \dots, k$ in case they have a negative reduced cost. In addition, we can limit the number of pricing problems that have to be solved at each CG iteration. As soon as no reduced cost column is found for a color i such that the corresponding constraint (6) is < 1 then no color $j > i$ can generate columns with negative reduced costs. Indeed, we would have $\pi_i = 0$ and $\pi_j = 0 \forall j > i$. Since the weights of the vertices in the maximum weight stable set pricing problems are

decreasing for increasing values of j , if there is no reduced cost variable for j , there cannot be for colors with a larger index.

In order to find an optimal integer solution of MIP_E , a branch-and-price algorithm is necessary, i.e., the CG procedure must be performed at each node of the branching scheme. The development of such an algorithm goes beyond the scope of this paper, we instead use this formulation to derive a heuristic algorithm and strong dual bounds for hard MCSP instances. Very good quality feasible integer solutions, can be obtained by solving MIP_E by considering only all the variables generated to optimally solve its LP relaxation. Computational tests show that the linear programming relaxation of MIP_E provides very tight Lower Bounds which allow us to solve to proven optimality some hard instances of the literature.

3 Computational results

In this section we present the results of preliminary computational experiments on two benchmark sets of instances. The first set is composed by random instances proposed by San Segundo [17] with 60, 70 and 75 vertices and density d varying from 0.1 to 0.9. For each value of d and n , we select one instance (27 instances in total). The second set is a subset of the DIMACS instances already used in the literature to compute upper bounds and lower bounds for MSCP (see [6]). We performed all experiments on a icore 7 computer at 3.4 GHz equipped with 16 GB RAM and using CPLEX 12.6 as MIP solver (with default parameters and in single thread mode). In the tables for each instance we report: the name of the instance, the number of vertices n , the number of edges m , the density d and the best lower and upper bounds (LB_l , UB_l) known in the literature for the DIMACS instances ([6]). A “tl” indicates that the time limit was reached for an instance and a “*” indicates that the value could not be computed (within the time limit). Bold UB values indicate that the previously unsolved instance has been solved to proven optimality. We used $k = \Delta(G) + 1$ as an upper bound on the strength of the graph. To solve the pricing problems, we used the Branch-and-Bound algorithm proposed by Held, Cook and Sewell [3]. CPLEX 12.6 was used to solve the linear programming relaxations at each iteration of the column generation procedure.

Table 1 compares the results of the two formulations MIP_C and MIP_E on the first benchmark of 27 instances. For MIP_C , we report: the value of the linear programming relaxation LB_C^{LP} and the associated computation time t_C^{LP} ; the value LB_C^r of the linear programming relaxation strengthened by cuts generated by CPLEX at the root node and the corresponding computation time

t_C^r . Imposing a time limit of 600 seconds, we report the values of the best lower and upper bounds (LB_C , UB_C) and finally the total computation time t_C . For MIP_E (imposing the same maximum time limit of 600 seconds), we report: the value LB_E^{LP} of the linear programming relaxation; the value UB_E is obtained by solving, with CPLEX, MIP_E restricted to the columns generated at the root node; the computation time t_E^r of the column generation process; the total computation time t_E and the number (*cols*) of generated columns. MIP_C is able to solve to optimality only 3 instances with $d = 0.1$. For all instances apart of density $d = 0.1$, MIP_E is able to produce a better lower bound than MIP_C . MIP_C obtains better UBs on instances with density $d \leq 0.5$ while MIP_E obtains better UBs for all other densities. The upper and lower bounds based on MIP_E are able to solve to optimality 2 instances (r70_80.1 and r75_90.1).

In Table 2 we report the results for DIMACS instances solved by MIP_C within the time limit (40 instances in total). We report in column $\Sigma(G)$ the chromatic sum of the instance. Compared to the best results from the literature (see [6]), MIP_C is able to solve to optimality, within 600 seconds, 21 previously unsolved instances. Most instances are solved within 10 seconds (27 instances), hence we focus on the remaining instances for the rest of our computational results, i.e., unsolved instances and instances solved by MIP_C in more than 10 seconds.

In Table 3, we present the results for seven hard DIMACS instances for which the linear programming relaxation of MIP_E has been successfully computed. Five previously unsolved instances have been solved to optimality by the lower and upper bounds based on MIP_E in less than 140 seconds. The remaining 2 instances have a very small optimality gap. For these 7 instances, MIP_C is only able to solve two of them in a much larger computation time.

4 Conclusions

In this manuscript, we have studied and tested two ILP formulations for MSCP. The MIP_C can be successfully solved via a commercial MIP solver in short computational time for a large subset of DIMACS instances from the literature. In addition thanks to the high quality of the lower and upper bounds based on MIP_E , we have been able to solve to proven optimality some additional instances where MIP_C fails. Thanks also to preliminary computational results on random instances, we can conclude that MIP_E is a very promising model to develop an exact branch-and-price algorithm for MSCP.

Instance				MIP_C							MIP_E				
name	n	m	d	LB_C^{LP}	t_C^{LP}	LB_C^r	t_C^r	LB_C	UB_C	t_C	LB_E^{LP}	UB_E	t_E^r	t_E	cols
r60_10.1	60	184	0.1	90	0.0	114	0.1	121	121	1.4	121	146	3.7	12.9	1585
r60_20.1	60	349	0.2	90	0.0	138	0.5	163	165	tl	161	192	2.7	8.4	1371
r60_30.1	60	513	0.3	90	0.0	159	1.7	181	199	tl	193	219	1.6	5.4	1638
r60_40.1	60	697	0.4	90	0.0	187	3.4	205	260	tl	241	269	1.0	22.0	1599
r60_50.1	60	863	0.5	90	0.1	221	4.8	235	305	tl	286	301	0.9	13.8	1880
r60_60.1	60	1053	0.6	90	0.1	270	10.7	282	371	tl	355	364	0.5	1.2	1978
r60_70.1	60	1257	0.7	90	0.1	344	18.7	350	464	tl	448	457	0.3	1.5	2279
r60_80.1	60	1397	0.8	90	0.1	426	32.6	434	576	tl	522	525	0.2	0.4	2289
r60_90.1	60	1600	0.9	90	0.2	624	58.2	628	693	tl	692	693	0.1	0.2	2141
r70_10.1	70	231	0.1	105	0.0	132	0.3	143	143	9.7	141	173	15.2	29.4	2145
r70_20.1	70	490	0.2	105	0.0	165	1.2	187	213	tl	202	286	9.0	22.6	1806
r70_30.1	70	729	0.3	105	0.0	199	2.3	217	272	tl	253	326	5.2	65.1	2315
r70_40.1	70	937	0.4	105	0.1	228	5.8	241	328	tl	306	364	2.6	62.5	2182
r70_50.1	70	1211	0.5	105	0.1	275	17.3	285	407	tl	372	416	1.8	61.7	2336
r70_60.1	70	1459	0.6	105	0.1	338	18.2	345	512	tl	461	477	1.3	61.2	2702
r70_70.1	70	1685	0.7	105	0.2	401	31.6	406	615	tl	552	565	0.9	9.8	2856
r70_80.1	70	1898	0.8	105	0.2	512	53.3	516	754	tl	647	647	0.5	0.7	3030
r70_90.1	70	2155	0.9	105	0.3	732	tl	732	1123	tl	866	868	0.2	0.6	2749
r75_10.1	75	287	0.1	112.5	0.0	147	0.2	167	167	115.2	162	2850	33.9	93.8	2377
r75_20.1	75	550	0.2	112.5	0.0	179	1.8	200	234	tl	219	320	15.6	75.5	2264
r75_30.1	75	839	0.3	112.5	0.0	217	5.8	227	303	tl	279	352	7.2	67.2	2594
r75_40.1	75	1117	0.4	112.5	0.1	253	12.5	262	399	tl	349	410	4.2	64.1	2871
r75_50.1	75	1366	0.5	112.5	0.2	303	21.1	314	497	tl	420	456	2.8	62.7	3039
r75_60.1	75	1663	0.6	112.5	0.2	365	27.4	369	585	tl	512	533	1.4	61.4	2970
r75_70.1	75	1907	0.7	112.5	0.2	434	43.5	437	704	tl	594	608	1.0	7.5	3216
r75_80.1	75	2234	0.8	112.5	0.4	610	110.2	613	886	tl	784	789	0.7	1.2	3367
r75_90.1	75	2478	0.9	112.5	0.5	809	tl	816	1231	tl	1037	1037	0.4	0.6	4193

Table 1

Comparison between MIP_C and MIP_E on random instances

References

- [1] Y. Alavi, P. Erdős, P.J. Malde, A.J. Schwenk and C. Thomassen: Tight bounds on the chromatic sum of a connected graph. *J. Graph Theor.*, 13(33): 353–357 (1989).
- [2] D. Cornaz, F. Furini, and E. Malaguti. Solving coloring problems as maximum weight stable set problems. *Discrete Appl. Math.*, (in press).
- [3] S. Held, W. Cook and E.C. Sewell: Maximum-weight stable sets and safe lower bounds for graph coloring. *Mathematical Programming Computation*, 4(4): 363–381 (2012).
- [4] A. Helmar and M. Chiarandini: A local search heuristic for chromatic sum. *Comput. Oper. Res.*, 39(7): 1724–1733 (2012).
- [5] Y. Jin, J.-K. Hao and J.P. Hamiez: A memetic algorithm for the Minimum Sum Coloring Problem. *Comput. Oper. Res.*, 43: 318–327 (2014).

Instance						MIP_C					
name	n	m	d	LB_l [6]	UB_l [6]	LB_C^{LP}	t_C^{LP}	LB_C^r	t_C^r	$\Sigma(G)$	t_C
2-Insertions3	37	72	0.1	55	62	56	0.0	58	0.0	62	0.2
3-Insertions3	56	110	0.1	84	92	84	0.0	87	0.0	92	0.2
anna	138	493	0.1	273	276	199	0.0	276	0.2	276	0.2
david	87	406	0.1	234	237	127	0.1	237	0.1	237	0.1
fpsol2.i.1	496	11654	0.1	3403	3403	631	5.3	3403	18.5	3403	19.6
fpsol2.i.2	451	8691	0.1	1668	1668	633	7.0	1668	19.1	1668	19.2
fpsol2.i.3	425	8688	0.1	1636	1636	607	6.2	1636	19.2	1636	19.2
games120	120	638	0.1	442	443	180	0.0	442	0.3	443	0.4
homer	561	1629	≈ 0.0	1129	1150	780	0.3	1146	2.2	1150	58.6
huck	74	301	0.1	243	243	111	0.0	243	0.1	243	0.1
inithx.i.1	864	18707	0.1	3676	3676	1124	22.3	3676	76.2	3676	81.4
inithx.i.2	645	13979	0.1	2050	2050	924	18.1	2050	60.2	2050	63.8
inithx.i.3	621	13969	0.1	1986	1986	901	21.4	1986	62.3	1986	62.9
jean	80	254	0.1	216	217	115	0.0	217	0.0	217	0.0
miles1000	128	3216	0.4	1623	1666	192	0.7	1657	11.1	1666	115.7
miles1500	128	5198	0.6	3239	3354	192	1.5	3326	10.8	3354	11.9
miles250	128	387	≈ 0.0	318	325	190	0.0	325	0.1	325	0.1
miles500	128	1170	0.1	686	705	192	0.1	702	0.8	705	17.4
miles750	128	2113	0.3	1145	1173	192	0.4	1157	3.6	1173	77.0
mug1001	100	166	≈ 0.0	188	202	150	0.0	195	0.1	202	0.9
mug10025	100	166	≈ 0.0	186	202	150	0.0	194	0.1	202	2.1
mug881	88	146	≈ 0.0	164	178	132	0.0	173	0.1	178	0.8
mug8825	88	146	≈ 0.0	162	178	132	0.0	173	0.1	178	0.6
mulsol.i.1	197	3925	0.2	1957	1957	266	1.2	1957	1.7	1957	1.7
mulsol.i.2	188	3885	0.2	1191	1191	275	1.8	1191	2.3	1191	2.3
mulsol.i.3	184	3916	0.2	1187	1187	271	2.1	1187	2.3	1187	2.3
mulsol.i.4	185	3946	0.2	1189	1189	273	1.8	1189	2.4	1189	2.4
mulsol.i.5	186	3973	0.2	1160	1160	274	2.0	1160	2.5	1160	2.5
myciel3	11	20	0.4	16	21	17	0.0	20	0.0	21	0.0
myciel4	23	71	0.3	34	45	35	0.0	39	0.0	45	0.3
myciel5	47	236	0.2	70	93	71	0.0	81	1.0	93	1.0
myciel6	95	755	0.2	142	189	143	0.1	162	2.8	189	10.7
qg.order30	900	26100	0.1	13950	13950	1350	2.5	13950	258.1	13950	271.4
queen5.5	25	160	0.5	75	75	38	0.0	75	0.0	75	0.0
queen6.6	36	290	0.5	126	138	54	0.0	126	0.2	138	168.7
queen7.7	49	476	0.4	196	196	74	0.0	196	0.3	196	0.5
queen8.12	96	1368	0.3	624	624	144	0.1	624	1.0	624	2.0
zeroin.i.1	211	4100	0.2	1822	1822	274	0.6	1822	2.1	1822	2.2
zeroin.i.2	211	3541	0.2	1004	1004	290	0.7	1004	2.1	1004	2.1
zeroin.i.3	206	3540	0.2	998	998	285	0.7	998	2.1	998	2.1

Table 2
Computational results of MIP_C on DIMACS instances

- [6] Y. Jin, J-P. Hamiez and J.-K. Hao: Algorithms for the minimum sum coloring problem: a review. CoRR abs/1505.00449 (2015).
- [7] Y. Jin and J.-K. Hao: Hybrid evolutionary search for the minimum sum coloring problem of graphs. Inform. Sciences, 352: 15–34 (2015).
- [8] C. Koulamas: Decomposition and hybrid simulated annealing heuristics for the parallel-machine total tardiness problem. Nav. Res. Log., 44(1): 109–125 (1997).
- [9] E. Kubicka and A. Schwenk: An introduction to chromatic sums. Proceedings

Instance						MIP_C			MIP_E				
name	n	m	d	LB_l [6]	UB_l [6]	LB_C	UB_C	t_C	LB_E^{LP}	UB_E	t_E^r	t_E	cols
flat300200	300	21375	0.5	1531	3150	*	6334	tl	3150	3150	59.2	59.9	1246
miles1000	128	6432	0.8	1623	1666	1666	1666	115.7	1666	1666	29.7	31.8	1149
miles1500	128	5198	0.6	3239	3354	3354	3354	11.9	3354	3354	1.8	2.4	451
queen6.6	36	290	0.5	126	138	138	138	168.5	138	138	0.0	0.2	128
queen8.8	64	1456	0.7	288	291	288	291	tl	291	291	1.2	20.0	2181
DSJC125.9	125	6961	0.9	1691	2503	1100	3154	tl	2500	2503	3.9	5.3	220
DSJC250.9	250	27897	0.9	4311	8277	700	11494	tl	8236	8495	99.2	159.2	459

Table 3

Comparison between MIP_C and MIP_E on DIMACS instances

of the 17th conference on ACM Annual Computer Science Conference (ACM): 39–45 (1989).

- [10] Y. Li, C. Lucet, A. Moukrim and K. Sghiouer: Greedy algorithms for the minimum sum coloring problem. Logistique et Transports Conference (2009).
- [11] E. Malaguti and P. Toth: A survey on vertex coloring problems. Intl. Trans. in Op. Res., 17(1): 1–34 (2010).
- [12] A. Mehrotra and M. Trick. A column generation approach for graph coloring. INFORMS J. Computing, 8(4): 344–54 (1996).
- [13] H. Bouziri and M. Jouini: Lower bounds for the minimal sum coloring problem. ELelectronic. Notes Discrete Math., 36(1): 663–670 (2010).
- [14] J. Mitchem and P. Morriss: On the cost-chromatic number of graphs. Discrete Math., 171(1–3): 201–211 (1997).
- [15] M.R. Salavatipour: On sum coloring of graphs. Discrete Appl. Math., 127(3): 477–488 (2003).
- [16] A. Sen, H. Deng and S. Guha: On a graph partition problem with application to VLSI layout. Inform. Process. Lett., 43(2): 87–94 (1992).
- [17] P. San Segundo: A new DSATUR-based algorithm for exact vertex coloring. Comput. Oper. Res., 39(7): 1724–1733 (2012).
- [18] Q. Wu and J.-K. Hao: An effective heuristic algorithm for sum coloring of graphs. Comput. Oper. Res., 39(7): 1593–1600 (2012).
- [19] Q. Wu and J.-K. Hao: Improved lower bounds for sum coloring via clique decomposition. CoRR abs/1303.6761 (2013).